

Лабораторна робота №1

Асинхронна обробка даних у TypeScript

1. Мета роботи

Метою лабораторної роботи є налаштування середовища розробки та набуття практичних навичок асинхронного програмування в TypeScript, зокрема:

- встановлення Node.js та налаштування TypeScript-проєкту;
- використання `Promise.all` та `Promise.race` для керування паралельними операціями;
- реалізація механізмів повторних спроб (`retry`) при помилках;
- послідовна обробка даних партіями (`batch processing`);
- коректна обробка помилок через `try/catch` в асинхронному коді.

2. Теоретичні відомості

2.1. Встановлення Node.js

Node.js — це середовище виконання JavaScript поза браузером, побудоване на рушії V8 від Google. Для встановлення рекомендується використовувати LTS-версію (Long Term Support) з офіційного сайту <https://nodejs.org>.

Після встановлення в терміналі доступні команди:

```
node --version    # версія Node.js
npm --version     # версія пакетного менеджера npm
```

Разом із Node.js встановлюється npm (Node Package Manager) — менеджер пакетів для встановлення бібліотек та інструментів.

2.2. Налаштування TypeScript-проєкту

TypeScript — це надмножина JavaScript, що додає статичну типізацію. Код TypeScript компілюється у звичайний JavaScript перед виконанням.

Ініціалізація нового проєкту:

```
npm init -y          # створити package.json
npm install -D typescript # встановити TypeScript як dev-залежність
npm run tsc -- --init  # створити tsconfig.json
```

Додавання скрипту збірки в package.json:

```
{
  "scripts": {
    "build": "tsc"
  }
}
```

Компіляція та запуск:

```
npm run build      # компілює .ts → .js
node dist/main.js  # запускає скомпільований файл
```

2.3. Promises в JavaScript та TypeScript

Promise — це об'єкт, що представляє результат асинхронної операції, який буде доступний у майбутньому. Promise може перебувати в одному з трьох станів:

- **pending** — очікування результату;
- **fulfilled** — операція завершилася успішно;
- **rejected** — операція завершилася помилкою.

Створення Promise:

```
const promise = new Promise((resolve, reject) => {  
  // Асинхронна логіка  
  setTimeout(() => resolve("Результат"), 1000);  
});
```

2.4. Async/Await

Конструкція `async/await` — це синтаксичний цукор над Promise, що дозволяє писати асинхронний код у синхронному стилі:

```
async function fetchData(): Promise<string> {  
  const result = await someAsyncOperation();  
  return result;  
}
```

Ключове слово `async` перед функцією гарантує, що вона завжди повертає Promise.

Ключове слово `await` зупиняє виконання функції до завершення Promise.

2.5. Promise.all та Promise.race

`Promise.all` приймає масив Promise і повертає новий Promise, який:

- виконується успішно, коли **всі** Promise завершилися успішно (повертає масив результатів);
- завершується з помилкою, якщо **хоча б один** Promise завершився з помилкою.

```
const results = await Promise.all([  
  fetchUser(1),  
  fetchUser(2),  
  fetchUser(3),  
]);  
// results: [User, User, User]
```

`Promise.race` повертає результат того `Promise`, який завершився **першим** (успішно або з помилкою):

```
const fastest = await Promise.race([
  fetchFromServer1(),
  fetchFromServer2(),
]);
```

3. Завдання

3.1. Встановлення Node.js

Встановити Node.js на робочий комп'ютер.

Вимоги:

- Завантажити LTS-версію з <https://nodejs.org>.
- Переконавшись, що команди `node --version` та `npm --version` працюють у терміналі.

3.2. Налаштування проєкту з TypeScript

Створити новий проєкт та налаштувати TypeScript.

Вимоги:

- Ініціалізувати проєкт: `npm init -y`.
- Встановити TypeScript: `npm install -D typescript`.
- Створити `tsconfig.json` через `npx tsc --init`.
- Створити папку `src/` для вихідного коду.
- Додати скрипт `"build": "tsc"` у `package.json`.
- Переконавшись, що `npm run build` компілює файли без помилок.

3.3. Функція `delay`

Сигнатура:

```
delay(ms: number): Promise<void>
```

Створює Promise, що завершується через вказаний час. Використовується як допоміжна функція в інших завданнях.

Вимоги:

- Приймає кількість мілісекунд затримки.
- Повертає `Promise<void>`, що завершується через `ms` мілісекунд.
- Використати конструктор `new Promise` та `setTimeout`.

Приклад використання:

```
await delay(1000); // чекаємо 1 секунду
console.log('Готово');
```

3.4. Функція `fetchUserProfiles`

Сигнатура:

```
fetchUserProfiles(userIds: string[]): Promise<UserProfile[]>
```

Інтерфейс:

```
interface UserProfile {
  id: string;
  name: string;
  email: string;
}
```

Симулює паралельне завантаження профілів користувачів з сервера.

Вимоги:

- Кожен запит має симулювати затримку 50–150 мс (випадково).
- Використати `Promise.all` для паралельного завантаження.
- Якщо `userIds` порожній — повернути порожній масив.
- Всі змінні та параметри мають бути типізовані.

Приклад використання:

```
const profiles = await fetchUserProfiles(['1', '2', '3']);
// Паралельне завантаження ~100мс, не ~300мс
console.log(profiles);
// [
//   { id: '1', name: 'User 1', email: 'user1@example.com' },
//   { id: '2', name: 'User 2', email: 'user2@example.com' },
//   { id: '3', name: 'User 3', email: 'user3@example.com' }
// ]
```

3.5. Функція `retryOperation`

Сигнатура:

```
retryOperation(  
  operation: () => Promise<unknown>,  
  maxRetries: number  
): Promise<unknown>
```

Виконує асинхронну операцію з автоматичними повторними спробами при помилці.

Вимоги:

- При помилці — зачекати 100 мс, спробувати знову.
- Максимум `maxRetries` спроб (за замовчуванням 3).
- Якщо всі спроби провалилися — викинути останню помилку.
- Логувати кожну спробу: `console.log('Спроба ${n}...')`.

Приклад використання:

```
let attempts = 0;  
const unreliableOperation = async (): Promise<string> => {  
  attempts++;  
  if (attempts < 3) {  
    throw new Error('Тимчасова помилка');  
  }  
  return 'Успіх!';  
};  
  
const result = await retryOperation(unreliableOperation, 3);  
// Лог: Спроба 1... → помилка → чекаємо 100мс  
//      Спроба 2... → помилка → чекаємо 100мс  
//      Спроба 3... → успіх!  
console.log(result); // 'Успіх!'
```

3.6. Функція `processInBatches`

Сигнатура:


```
processInBatches(
  items: number[],
  batchSize: number,
  processor: (batch: number[]) => Promise<number[]>
): Promise<number[]>
```

Обробляє великий масив елементів партіями (batches) послідовно.

Вимоги:

- Розділити `items` на партії розміром `batchSize`.
- Обробити партії **послідовно** (не паралельно!).
- Кожну партію обробити функцією `processor`.
- Повернути масив усіх результатів.
- Логувати прогрес: `console.log('Обробка партії ${n}/${total}...')`.

Приклад використання:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];

const processor = async (batch: number[]): Promise<number[]> => {
  await delay(100); // Симуляція обробки
  return batch.map(n => n * 2);
};

const results = await processInBatches(numbers, 3, processor);
// Партія 1: [1,2,3] → [2,4,6] → чекаємо
// Партія 2: [4,5,6] → [8,10,12] → чекаємо
// Партія 3: [7,8] → [14,16] → готово
console.log(results); // [2, 4, 6, 8, 10, 12, 14, 16]
```

3.7. Функція `raceWithTimeout`

Сигнатура:

```
raceWithTimeout(
  promise: Promise<unknown>,
  timeoutMs: number
): Promise<unknown>
```

Виконує `Promise` з обмеженням часу виконання.

Вимоги:

- Використати `Promise.race`.
- Якщо `promise` не завершився за `timeoutMs` — викинути помилку.
- Текст помилки: `'Operation timed out after ${timeoutMs}ms'`.

Приклад використання:

```
// Швидка операція – успіх
const fast = delay(50).then(() => 'Готово');
const result1 = await raceWithTimeout(fast, 100);
console.log(result1); // 'Готово'

// Повільна операція – timeout
const slow = delay(200).then(() => 'Готово');
try {
  await raceWithTimeout(slow, 100);
} catch (err) {
  if (err instanceof Error) {
    console.error(err.message); // 'Operation timed out after 100ms'
  }
}
```

4. Загальні вимоги

1. Використовувати **TypeScript** з типізацією всіх функцій.
2. Налаштувати `tsconfig.json`.
3. Всі функції мають використовувати `async/await`.
4. Обробка помилок через `try/catch`.
5. Логування прогресу виконання.
6. Читабельний та структурований код.
7. Експортувати всі функції з одного файлу.
8. Підготувати звіт з поясненням роботи кожної функції.
9. Завантажити код та звіт на `GitLab` та надати доступ викладачам.

5. Приклад CLI-програми

Для тестування реалізованих функцій створіть файл `main.ts` з наступним кодом:

```
async function main(): Promise<void> {
  console.log('=== Тест 1: Паралельне завантаження ===');
  console.time('fetchUserProfiles');
  const profiles = await fetchUserProfiles(['1', '2', '3', '4', '5']);
  console.timeEnd('fetchUserProfiles');
  console.log(`Завантажено ${profiles.length} профілів`);

  console.log('\n=== Тест 2: Retry механізм ===');
  let attempt = 0;
  const result = await retryOperation(async () => {
    attempt++;
    if (attempt < 2) throw new Error('Помилка');
    return 'Успіх!';
  }, 3);
  console.log(result);

  console.log('\n=== Тест 3: Batch обробка ===');
  const items: number[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
  const processed = await processInBatches(
    items,
    3,
    async (batch: number[]) => {
      await delay(50);
      return batch.map(n => n * 2);
    }
  );
  console.log('Оброблено:', processed);
  console.log('\n=== Тест 4: Timeout ===');
  try {
    await raceWithTimeout(delay(200), 100);
  } catch (err) {
    if (err instanceof Error) {
      console.error('Timeout:', err.message);
    }
  }
}
```

```
main().catch(console.error);
```